

Programação Orientada a Objetos em TypeScript

Guia de estudo — classes, encapsulamento, herança, interfaces e quando usar POO na prática

1. Classes

Uma classe é um molde para criar objetos, agrupando dados (propriedades) e comportamentos (métodos) relacionados. No exemplo abaixo, a classe **Aluno** guarda nome, matrícula e IRA, além de expor métodos para interagir com esses dados.

```
class Aluno {
  nome: string
  matricula: number
  private ira: number // "private" só pode ser acessado dentro da própria classe

  constructor(nome: string, matricula: number, ira: number) {
    this.nome = nome
    this.matricula = matricula
    this.ira = ira
  }

  mostrarInfo(): string {
    return `${this.nome} (matrícula: ${this.matricula})`
  }

  getIra(): number {
    return this.ira
  }
}

const aluno1 = new Aluno("Isac", 20261148060040, 89.2)
console.log(aluno1.mostrarInfo())
console.log(aluno1.getIra())
```

2. Encapsulamento (public, private, protected)

Encapsulamento é o princípio de esconder detalhes internos de um objeto, expondo apenas o que é necessário. Em TypeScript isso é controlado por modificadores de acesso:

```
class ContaBancaria {
  private saldo: number = 0 // só acessível dentro da própria classe

  depositar(valor: number): void {
    this.saldo += valor
  }

  consultarSaldo(): number {
    return this.saldo
  }
}
```

```

}

const conta = new ContaBancaria()
conta.depositar(100)
console.log(conta.consultarSaldo()) // funciona
// console.log(conta.saldo)         // erro! "saldo" é privado

```

Modificador	Onde pode ser acessado
public (padrão)	De qualquer lugar
private	Somente dentro da própria classe
protected	Na própria classe e em classes filhas (herança)

3. Herança

Herança permite que uma classe (filha) reaproveite propriedades e métodos de outra classe (pai), podendo estender ou sobrescrever seu comportamento.

```

class Pessoa {
  constructor(public nome: string) {} // atalho: já declara e atribui a propriedade

  apresentar(): string {
    return `Meu nome é ${this.nome}`
  }
}

class Estudante extends Pessoa {
  constructor(nome: string, public curso: string) {
    super(nome) // chama o construtor da classe pai
  }

  apresentar(): string {
    return `${super.apresentar()} e curso ${this.curso}` // reaproveita o método do pai
  }
}

const est = new Estudante("Isac", "TSI")
console.log(est.apresentar())

```

4. Interfaces (contratos, não geram código)

Uma *interface* define um contrato de tipo: quais propriedades e métodos uma classe deve possuir, sem implementar nada. Diferente de uma classe, a interface é apagada durante a transpilação — ela não vira código JavaScript real.

```

interface Animal {
  nome: string
  emitirSom(): string
}

```

```
class Cachorro implements Animal {
  constructor(public nome: string) {}

  emitirSom(): string {
    return "Au au!"
  }
}
```

5. Classes abstratas

Uma classe abstrata não pode ser instanciada diretamente — ela serve como base para outras classes, podendo definir métodos obrigatórios (abstratos) que as classes filhas são forçadas a implementar.

```
abstract class Forma {
  abstract calcularArea(): number // obrigatório implementar nas classes filhas

  descrever(): string {
    return `Área: ${this.calcularArea()}`
  }
}

class Retangulo extends Forma {
  constructor(private largura: number, private altura: number) {
    super()
  }

  calcularArea(): number {
    return this.largura * this.altura
  }
}

// const forma = new Forma() // erro! não dá pra instanciar uma classe abstrata direto
const retangulo = new Retangulo(5, 3)
console.log(retangulo.descrever())
```

6. Polimorfismo

Polimorfismo é a capacidade de tratar objetos de classes diferentes de forma genérica (por exemplo, todos como "Pessoa"), enquanto cada um executa sua própria versão de um método.

```
function apresentarTodos(pessoas: Pessoa[]) {
  pessoas.forEach(p => console.log(p.apresentar()))
}
```

7. POO é muito usado em TypeScript?

A resposta depende bastante do contexto em que o TypeScript está sendo usado.

Onde POO é bem comum

- **Angular** (framework de frontend): usa POO pesadamente — componentes e serviços são baseados em classes com decorators.
- **NestJS** (framework de backend muito popular): estruturado inteiramente em classes, injeção de dependência e decorators, no estilo Java/C#.
- **TypeORM / Prisma** (bancos de dados): modelos de dados costumam ser definidos como classes.

Onde o estilo funcional domina

- **React** (o framework de frontend mais usado hoje): desde os Hooks (2019), a comunidade migrou para componentes funcionais. Classes são raras em código React moderno.
- **Node.js puro / Express**: muitos projetos organizam o código com funções e módulos, em vez de classes, especialmente em APIs pequenas.
- Bibliotecas como **lodash** e operações como `map/filter/reduce` incentivam um estilo mais funcional (funções puras, imutabilidade).

Por que essa divisão existe

O JavaScript nasceu com um sistema de orientação a objetos baseado em **protótipos**, não em classes de verdade — a sintaxe `class` usada hoje é, por baixo dos panos, açúcar sintático sobre esse sistema de protótipos. Isso levou a comunidade JS a adotar bastante o estilo funcional historicamente. Já linguagens como Java, C# ou C++ nasceram desde o início pensadas para POO, por isso o costume é mais arraigado nelas.

Resumo por contexto

Contexto	POO é comum?
Frameworks enterprise (Angular, NestJS)	Muito
Modelagem de dados / entidades de banco	Bastante
React moderno (hooks)	Pouco
Scripts e APIs pequenas em Node	Varia, funcional é comum
Bibliotecas utilitárias (lodash, etc)	Pouco, tende ao funcional

Conclusão prática

Vale a pena aprender POO em TypeScript, porque você vai encontrá-la com certeza — principalmente ao trabalhar com Angular, NestJS ou modelagem de dados. Mas não se surpreenda se, no dia a dia, muito código TypeScript por aí (principalmente em React) for escrito num estilo mais funcional, com funções, `const`, arrow functions e hooks, em vez de `class`.